



Introduction to Functions in PHP



What is a Function?

A **function** in PHP is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, we define it once and call it whenever needed.

A function is an independent named block of code that is performed a specific task

PHP functions are similar to other programming languages.

A function is a piece of code which takes one more input in the form of parameter and does some processing and returns a value.

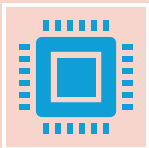
Advantages of PHP Functions:



Code Reusability: PHP functions are defined only once and can be invoked many times, like in other programming languages.



Less Code: It saves a lot of code because you don't need to write the logic many times. By the use of a function, you can write the logic only once and reuse it.



Easy to understand: PHP functions separate the programming logic. So it is easier to understand the flow of the application because every logic is divided in the form of functions.

How to create a function:

A few considerations must be made while developing a user-defined function:

- A function is any name that has an open and closed parenthesis at the end.
- The term function always appears at the start of a function name.
- To invoke a function, just write its name and the parentheses.
- You can't begin a function name with a number. It may start with an underscore or an alphabet.
- The case of a function name is irrelevant.

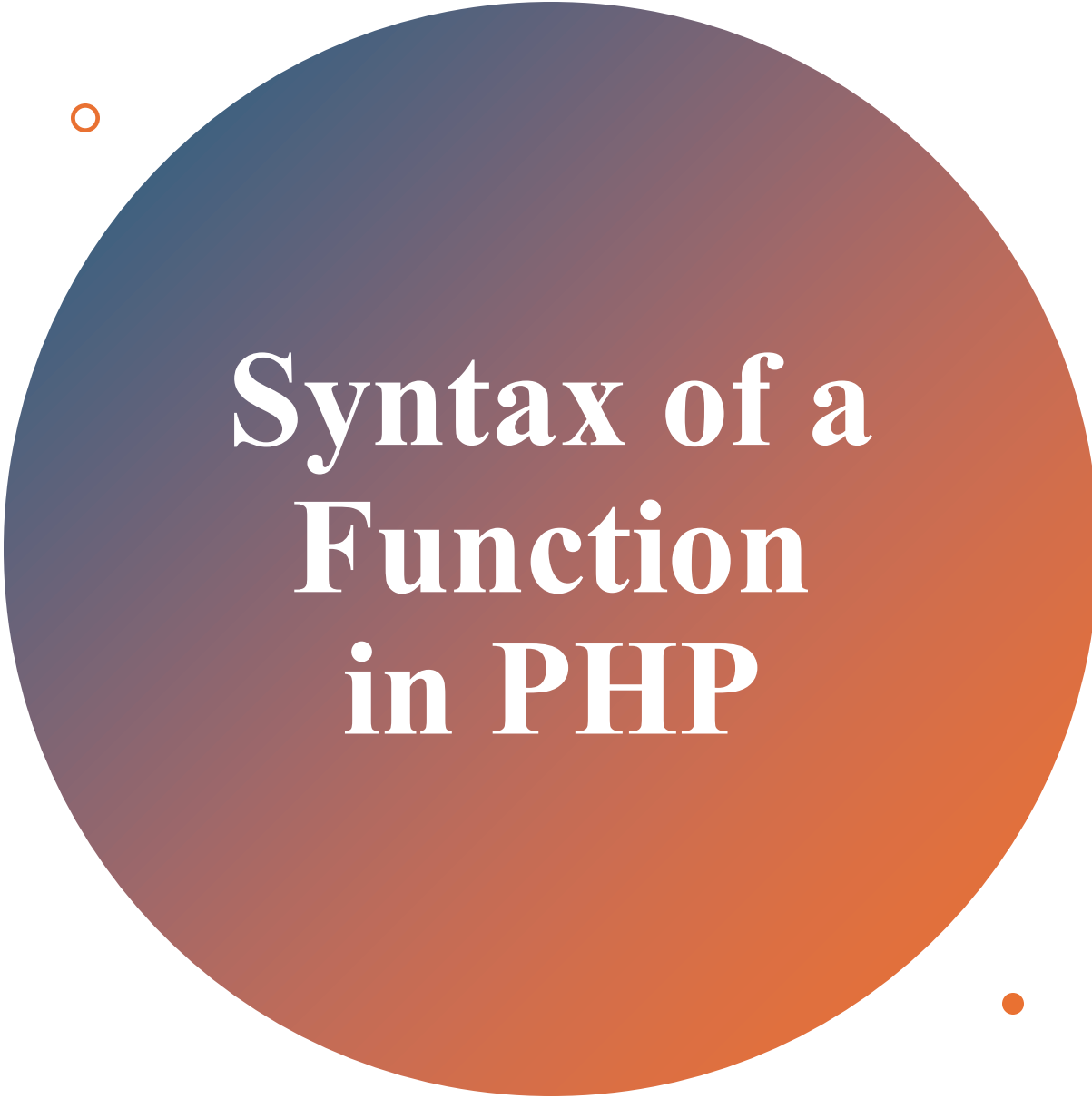
Types of Functions in PHP

PHP supports two types of functions:

1. **Built-in Functions** – PHP offers us a vast array of built-in library functions. These functions are already saved as functions and programmed.(e.g., `strlen()`, `array_push()`, `date()`)
2. **User-Defined Functions** – PHP enables us to develop our customized functions, known as user-defined functions, in addition to the built-in functions. This allows us to write our code packages, which we can then call as needed.



Syntax of a Function in PHP



```
function functionName() {  
    // Code to execute  
}
```

Calling the function:

- `functionName();` // This executes the function

Example: Simple Function

```
<?php  
  
function sayHello() {  
    echo "Hello, PHP!";  
}  
  
// Calling the function  
sayHello();  
  
?>
```

Output:

Hello, PHP!

Function Arguments or Parameters

PHP gives you option to pass your parameters inside a function.

You can pass as many as parameters you like.

These parameters work like variables inside your function.

Function functionName(parameter1,parameter2) {- --}

//function definition

//Function call

functionName(argument1,argument2);

Example

```
<?php
function addNumbers($a, $b) {
    return $a + $b;
}

// Calling the function
echo "Sum: " . addNumbers(5, 10);
?>
```

Function with Default Parameter

```
<?php
function greet($name = "Guest") {
    echo "Hello, $name!";
}

// Calling function with and without parameters
greet("John"); // Output: Hello, John!
greet();       // Output: Hello, Guest!
?>
```

Returning Values from Functions

Functions can return values using the return keyword.

```
<?php  
function multiply($x, $y) {  
    return $x * $y;  
}
```

```
$result = multiply(4, 5);  
echo "Product: $result";  
?>
```



In-Built Functions in PHP



PHP provides a vast library of **predefined (built-in) functions** to perform various tasks such as string manipulation, mathematical operations, array handling, file management, and more.

Types of In-Built Functions in PHP

PHP has several categories of built-in functions:

String Functions (Manipulate text)

- `strlen($str)` → Returns the length of a string
- `strtolower($str)` → Converts a string to lowercase
- `strtoupper($str)` → Converts a string to uppercase
- `strrev($str)` → Reverses a string
- `strpos($str, "word")` → Finds the position of a substring
- `str_replace("old", "new", $str)` → Replaces text

Example

```
<?php
$text = "Hello PHP!";
echo "Length: " . strlen($text) . "<br>";
echo "Uppercase: " . strtoupper($text) . "<br>";
echo "Reversed: " . strrev($text) . "<br>";
?>
```

Mathematical Functions (Perform calculations)

`abs($num)` → Absolute value

`pow($base, $exp)` → Exponentiation (power)

`sqrt($num)` → Square root

`round($num)` → Rounds a number

`rand(min, max)` → Generates a random number

```
<?php
echo "Absolute: " . abs(-10) .
"<br>";
echo "Power: " . pow(2, 3) .
"<br>";
echo "Square Root: " . sqrt(16) .
"<br>";
echo "Random: " . rand(1, 100) .
"<br>";
?>
```

Array Functions (Work with arrays)

- `count($array)` → Counts elements in an array
- `array_push($array, $val)` → Adds an element to the end
- `array_pop($array)` → Removes the last element
- `array_merge($arr1, $arr2)` → Merges two arrays
- `sort($array)` → Sorts an array in ascending order

```
<?php
$fruits = ["Apple", "Banana", "Cherry"];
array_push($fruits, "Mango");
sort($fruits);

echo "Total Fruits: " . count($fruits) . "<br>";
print_r($fruits);
?>
```


Date & Time Functions (Work with dates)

- `date("Y-m-d")` → Current date
- `time()` → Current Unix timestamp
- `mktime(hour, min, sec, month, day, year)` → Creates a timestamp

```
<?php
echo "Today's Date: " . date("Y-m-d") .
"<br>";
echo "Current Time: " . date("H:i:s") .
"<br>";
?>
```

File Handling Functions (Work with files)

- `fopen("file.txt", "r")` → Opens a file
- `fwrite($file, "content")` → Writes to a file
- `fread($file, size)` → Reads from a file
- `fclose($file)` → Closes a file

```
<?php
$file = fopen("test.txt", "w");
fwrite($file, "Hello, this is PHP!");
fclose($file);
?>
```

Functions in PHP: Call by Value & Call by Reference

When passing arguments to functions in PHP, they can be passed in two ways:

1. **Call by Value** (default)
2. **Call by Reference**

Call by Value (Default Behavior)

A copy of the variable is passed to the function.

Changes made inside the function **do not** affect the original variable.

Example of Call by Value

```
<?php
function changeValue($num){
    $num = $num + 10;
    echo "Inside function: $num <br>";
}

$value = 20;
changeValue($value);
echo "Outside function: $value";
?>
```

Call by Reference (Using &)

The function gets a **reference** to the original variable.

Changes made inside the function **affect the original variable**.

```
<?php
```

```
function changeValueRef(&$num) {
```

```
    $num = $num + 10;
```

```
    echo "Inside function: $num <br>";
```

```
}
```

```
$value = 20;
```

```
changeValueRef($value);
```

```
echo "Outside function: $value";
```

```
?>
```

When to Use Call by Value vs Call by Reference?

Feature	Call by Value	Call by Reference
Default Behavior	 Yes	 No (Uses &)
Copies Data?	 Yes	 No
Affects Original Variable?	 No	 Yes
Use Case	When the function should not modify the original data	When the function needs to modify the original data

Example: Swapping Two Numbers (Call by Reference)

```
<?php
function swapNumbers(&$a, &$b) {
    $temp = $a;
    $a = $b;
    $b = $temp;
}

$x = 5;
$y = 10;
swapNumbers($x, $y);

echo "After Swap: x = $x, y = $y";

?>
```